Progress Report — Jyotir

AMS 560 · Big Data Systems, Algorithms and Networks · Fall 2026 · Group 20

1. Objective and Description of the Project

The project builds a scheduling system for AI inference requests sent to geo-distributed GPU nodes. When a user sends a request to an LLM, the system must decide which GPU node should handle it. Existing schedulers ignore live hardware conditions and route requests to nodes that are physically unable to serve them — the GPU memory may be full, the chip may be overheating, or the power draw may have hit its limit. Our system solves this with a two-stage approach.

The first stage is a **feasibility filter** that checks every node against hard physical rules and removes any node that cannot safely handle the request. The second stage is a **learning policy** (sliding-window LinUCB) that picks the best node from those that remain, and improves its choices over time based on observed outcomes. The system is tested across six change scenarios — stable operation, a traffic burst, an electricity price shift, a thermal throttle event, a network outage, and a node failure and recovery — to measure how quickly it adapts after something in the cluster changes.

My role is to build the feasibility filter and five static baseline scheduling policies. The filter is the first thing every request hits; it enforces safety and ensures the learning policy only ever picks from nodes that can actually serve the request. The baselines are simple comparison policies — they let us measure how much benefit the learned policy adds over basic rules like "pick the closest node" or "pick the least busy node."

2. Timeline and Expected Deliverables

My assigned deliverables for this project are:

- Feasibility filter module with five safety rules and a YAML config file.
- Six baseline scheduling policies: Round-Robin, Min RTT, Least Load, Lowest Energy Cost, Static Weighted, and Power-of-Two.
- Stub objects so each module can be built and tested independently before the full team integrates.
- Edge-case test suite covering all known failure modes of the filter.
- Baseline decision logs: run all baselines on all six change scenarios and save the output logs that the experiment analysis will use for comparison.
- Report section covering the filter rules, rejection path, and all baseline policies.

3. How Much Has Been Completed

Feasibility filter — complete. The filter checks five rules in order: node status (must be up or warming), model availability (the requested model must be loaded), KV cache capacity (the request's tokens must fit in remaining GPU memory), power headroom (must be at least 2 kW above current draw), and temperature (must be below 85 °C). Every threshold lives in a YAML config file — nothing is hardcoded. Each rejected node gets a specific reason string so the router knows why it was removed. Warming nodes are admitted with a tag that downstream policies can use to apply a soft preference penalty, but they still go through every safety rule.

> *Could improve:* Four of the six baselines currently ignore the warming tag and treat a recovering node the same as a fully healthy one. A small load penalty for warming nodes in those policies could reduce the traffic burst that floods a returning node in Scenario S6.

Baseline policies — complete. All six policies are implemented. Round-Robin rotates through candidates in order. Min RTT picks the lowest-RTT node. Least Load picks the node with the fewest waiting and running requests. Lowest Energy picks the cheapest node by electricity price, skipping saturated ones. Static Weighted combines RTT, load, and price with fixed weights (0.5 / 0.3 / 0.2). Power-of-Two samples two candidates and picks the less busy one.

> *Could improve:* Least Load counts every request as equal weight regardless of token count — weighting by tokens would give a truer picture of node pressure. Static Weighted uses the same weights for all six scenarios; a short weight sweep before the final run could find a better default. Lowest Energy's saturation check requires the `capacity` field; a fallback for when it is missing would make it more robust.

Stub objects — complete. Synthetic node and request objects let the filter and baselines run without the simulator or router. Switching to real modules is a one-line import change per file.

Could improve: The fake cluster has five fixed nodes. More variety — different GPU types, mixed model sets, nodes near capacity limits — would make local testing more representative of the real simulator.

Edge-case tests — complete. 27 tests across six categories all pass: all nodes overheated, single feasible node, warming node admitted and tagged, warming node failing a safety rule still rejected, loose status values rejected, token-exact memory check.

Could improve: All tests run on four-to-five-node clusters. A stress test on a larger cluster (100+ nodes) would confirm the filter stays fast as the cluster scales.

Report section — complete. The written section covering filter rules, warming node logic, and all six baselines with per-scenario behavior tables is drafted and pushed.

Baseline decision logs — not yet complete. Running the baselines to produce decision logs requires the full end-to-end pipeline: Parikshit's simulator producing realistic traffic, routed through Krupesh's router service calling my filter and baselines, with the output written in the agreed log format. That pipeline is not yet fully wired. This is the only remaining deliverable.

4. Plan to Complete the Remaining Deliverable

The baseline decision logs require two things to happen first. The first is confirming that my filter and baselines import correctly through Krupesh's router instead of through the stubs — this is a two-line change per baseline file and should take under an hour once the router's adapter for my module is confirmed stable. The second is running the baselines through all six event traces (S1–S6) from Parikshit's simulator, which will produce one Parquet log file per policy per scenario. Those logs are the input Riya needs for every comparison figure in the final report.

Once the team confirms the end-to-end pipeline runs clean on one scenario, I can produce all six baseline log files in a single batch run. The plan is to do this as soon as the integration check passes, then hand the logs to Riya.

5. Difficulties

Building without the full system ready. My filter sits between the simulator (Parikshit) and the router (Krupesh), so I could not test the real end-to-end flow until both teammates' modules were stable. I solved this by building synthetic stub objects that match the agreed data format, which let me complete and test everything independently. The remaining risk is a last-minute schema change that requires re-running tests.

Memory check was not accurate enough. The first version of the filter rejected a node if its KV cache fill fraction was above 90 %. This turned out to be wrong: a node at 50 % fill with a total capacity of 1,000 tokens still cannot fit a request that needs 1,050 tokens, but the fraction check would pass it. The fix was to use the actual token capacity from the telemetry and check whether the request's tokens fit in the remaining space, with the fraction check kept as a fallback when capacity is not reported.

Warming nodes were skipping safety checks. An early version of the filter treated a warming node as a special case and returned it as allowed without checking model availability, memory, power, or temperature. A node that is warming up but also out of memory would have been incorrectly admitted. The fix was to run warming nodes through all five rules the same as any other node, and only tag them as warming at the very end if they pass everything. New tests were added to lock in this behavior.

All individual code deliverables are complete. The only open item is producing the decision logs once the full end-to-end pipeline is verified with the team.